



Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

AdaBoost

WEEK 2 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Side by side, or one after another

	Bagging and forests (week 1)	Boosting (week 2)
Rows each tree sees	its own bootstrap sample	every row, with weights
Order	independent	one after another
The trees	deep and unstable	small and weak: stumps
Combined by	a plain average	a weighted vote
What it fixes	the noise of one deep tree	the bias of one small tree

Each new tree is aimed at the rows the trees before it got wrong.

A stump: a tree with one question

The stump asks

PAY_0 <= 1.5

16.51%

default rate, yes side (16,115 rows)

70.13%

default rate, no side (1,885 rows)

0.6361

validation ROC AUC

Weak on purpose. AdaBoost uses two hundred of them.

Round one by hand

1. Fit

a stump on the weighted rows

2. Error

share of the weight on missed rows

3. Weight

$\ln((1 - e) / e)$, samme_alpha

4. Reweight

missed rows times e^{α} , then / sum

3,223

rows missed of 18,000

0.1791

weighted error e

1.5228

stump weight

Every row starts at $1/n$, so round one's weighted error is the share of rows missed.

After reweighting, the missed rows carry half the weight

0.1791

missed rows' weight, before

0.5000

missed rows' weight, after

4.5849

a missed row over a correct one

$e \times (1 - e) / e = 1 - e$ the missed rows now weigh what the correct rows weigh

On the new weights the last stump is a coin toss. The next stump has to find something else.

Three rounds by hand, matched to scikit-learn

Round	Question	Weighted error	Hand weight	scikit-learn
1	PAY_0 <= 1.5	0.1791	1.5228	1.5228
2	PAY_AMT1 <= 1223.5	0.4128	0.3526	0.3526
3	PAY_3 <= 1.5	0.4356	0.2592	0.2592

Equal within 1e-12. AdaBoostClassifier is these four steps, repeated.

```
ada_3.estimator_weights_    # each stump's weight in the vote
```

Two hundred stumps

0.7708

validation ROC AUC

0.5321

average precision

0.6361

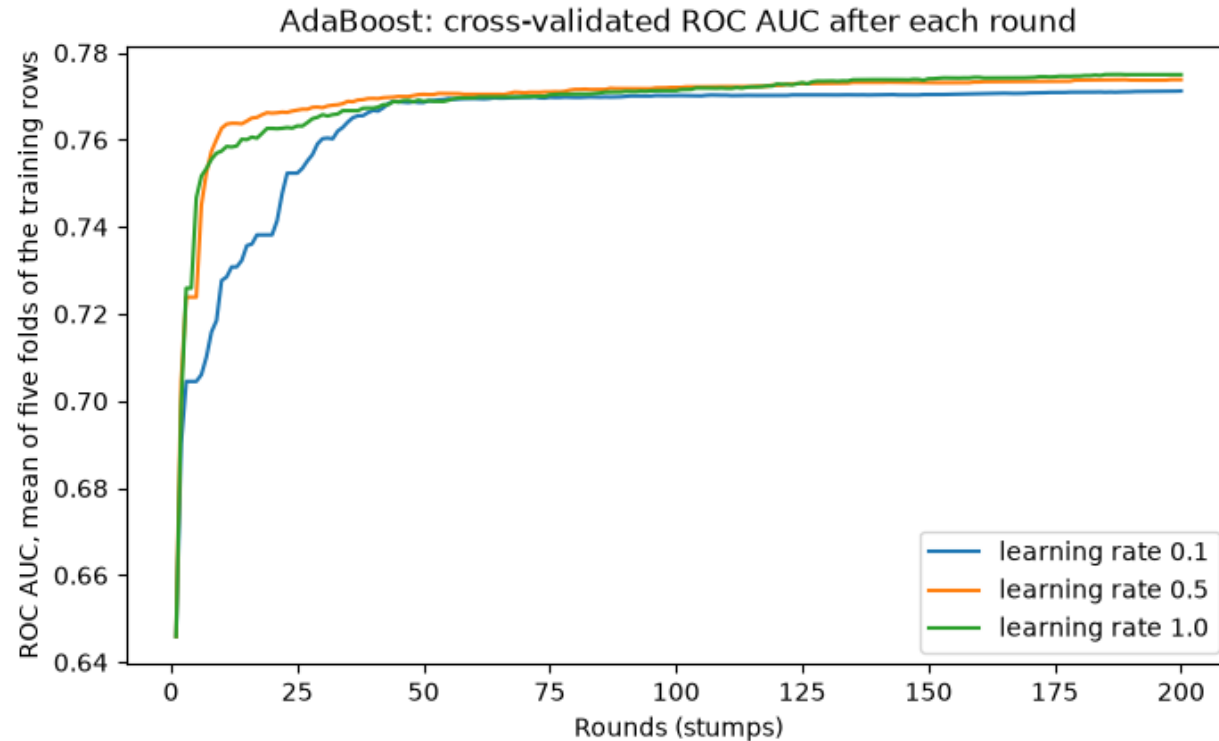
one stump

The stumps ask about 19 of the 19 columns

PAY_0 40 times, PAY_AMT3 28 times, BILL_AMT1 26 times

A sum of one-column steps: it bends along each column, never two at once.

The learning rate and the number of rounds



Rounds	0.1	0.5	1.0
10	0.7277	0.7626	0.7574
50	0.7686	0.7704	0.7688
200	0.7712	0.7738	0.7749

Rate 1.0 peaks at round 188, 0.7750. Five folds of the training rows; the validation rows play no part.

The learning rate mostly decides how many rounds it takes, not where the curves end.

A weighted vote, not a frequency

0.1192

lowest possible

0.8808

highest possible

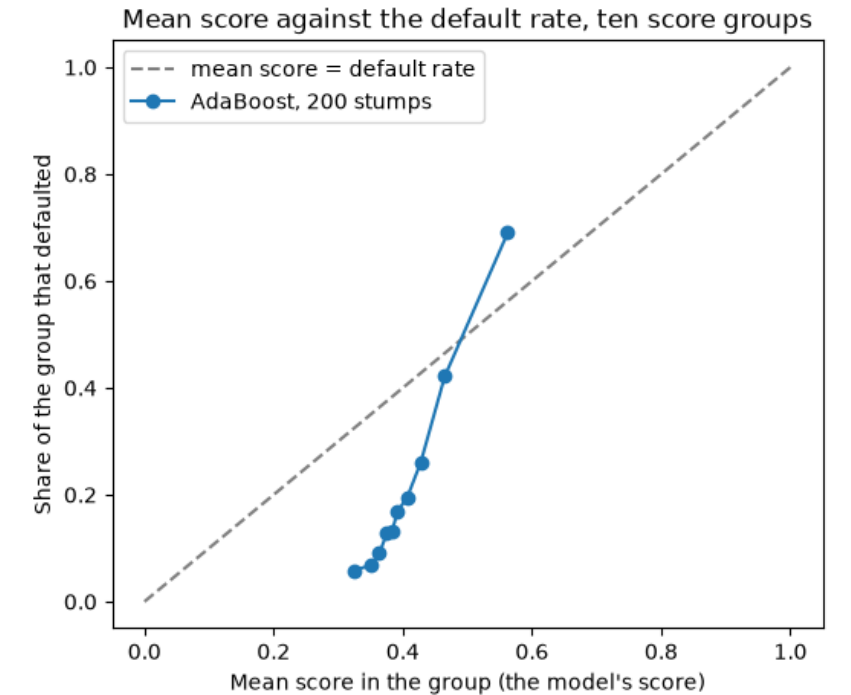
0.4051

mean output

0.2212

default rate

Score group	Mean score	Default rate
1	0.3244	0.0583
5	0.3832	0.1308
10	0.5623	0.6900



It ranks well; its level is not a share of defaults. The model's score, never a probability. Vote -0.4803 to 0.3243.

The rows that carry the weight after 50 rounds

Weights run from 0.29 to 6.63 times where they started. The rows at three times or more:

Rows	Count	Share of weight	Default rate	PAY_0 >= 2
Weight 3 x start or more	1,309	27.8%	93.4%	6.6%
All training rows	18,000	100.0%	22.1%	10.5%

Mostly accounts that defaulted while their latest status looked current: where the model struggles in this file, not a judgment of the accounts.

Where the week's name comes from

Yoav Freund and Robert E. Schapire, August 1997

"A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting"

Journal of Computer and System Sciences, volume 55, issue 1, pages 119 to 139

The last word of the title is the name of this week.

Source, read 5 October 2026: the Crossref record of the paper (DOI 10.1006/jcss.1997.1504). Bibliographic facts only; the paper itself was not read for this course.

Excel side by side

Excel side by side (tested in Excel)

`=1/ROWS($A$2:$A$18001)`

`=SUMPRODUCT(D2:D18001,A2:A18001)/SUM(D2:D18001)`

`=LN((1-M2)/M2)`

`=D2*EXP($M$3*A2)`

`=E2/SUM($E$2:$E$18001)`

`=SUMPRODUCT(F2:F18001,A2:A18001)`

the same on F and H with B and C

starting weights, D2 down

weighted error 0.1791

stump weight 1.5228

missed rows grow, E2 down

new weights, F2 down

missed rows' share 0.5000

rounds 2, 3: 0.3526, 0.2592

No AdaBoost in Excel; the stumps' missed columns come from Python. LOG is base 10: `=LOG(4)` gave 0.6021, `=LN(4)` gave 1.3863.

Save the day: git stash

1. Commit

mlkit.py, ensemblekit, tests

2. Log

results.csv at the out-of-fold
threshold

3. Stash

a learning-rate experiment, put
aside

4. Drop

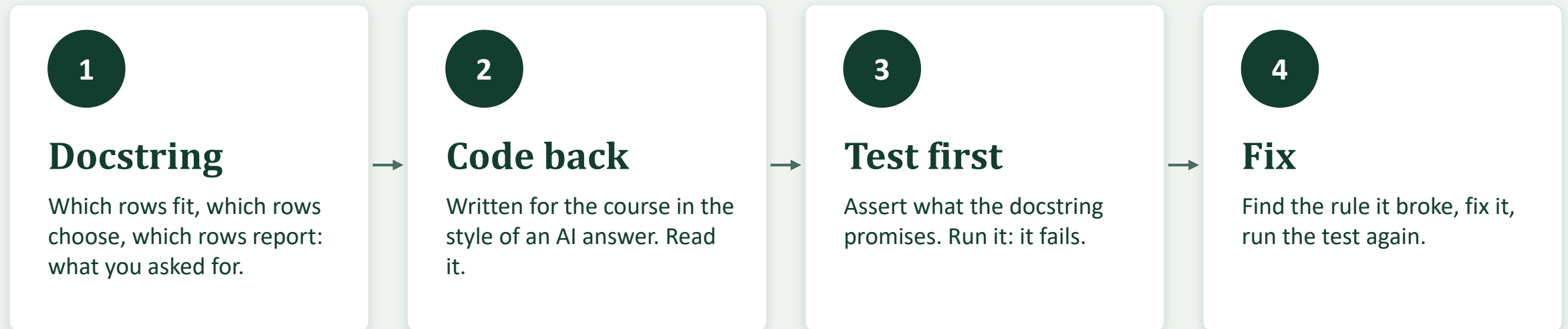
compare, then throw it away

```
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1
adaboost_200_stumps,validation,0.7708,0.5321,0.4009,0.7047,0.4021,0.6888,0.5078
+adaboost_200_stumps_rate_0.5,validation,0.7706,0.5327,0.3164,0.6980,0.3955,0.6918,0.5033
```

Threshold 0.4009, from out-of-fold scores, far from 0.5. Rate 0.5 gives 0.7706: stashed and dropped.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.



Today: the stump's weight is computed with `np.log10`: positive, plausible, and a learning rate nobody chose.

Code that runs is not the same as code that is right.

What you can do now

- Boosting: trees one after another, a weighted vote
- Run a round by hand: error, `same_alpha`, reweight
- The missed rows end with half the weight
- Compare settings inside the training rows
- AdaBoost's output is the model's score

Week 2, Day 2 Gradient boosting from residuals, on the benchmark bonds